Car Infotainment UPS Power Manager Documentation

Overview

This Python script manages automatic power control for a Raspberry Pi 4-based car infotainment system using the MakerFocus UPSPACK_V3 uninterruptible power supply. It ensures graceful shutdown when car power is disconnected and automatic startup when power is restored.

System Architecture

Components

- **Raspberry Pi 4**: Running the infotainment system (react-carplay)
- **UPSPACK_V3**: Provides battery backup and power management
- **Python Script**: Monitors power state and manages shutdowns
- **GPIO Communication**: Primary shutdown signal via GPIO18
- **UART Communication**: Optional detailed status monitoring

Code Breakdown

1. Initialization (`__init__`)

- Sets up logging to track system events
- Initializes GPIO pin 18 for shutdown signal detection
- Attempts UART connection for detailed monitoring
- Registers signal handlers for clean exit

2. Power State Management

- **PowerState Enum**: Tracks current system state
  - `EXTERNAL_POWER`: Car power connected
  - `BATTERY_POWER`: Running on UPS battery
  - `LOW_BATTERY`: Battery below 10%
  - `CRITICAL_BATTERY`: Battery below 5%
  - `SHUTTING_DOWN`: System shutdown in progress

3. Key Functions

- **`monitor_loop()`**: Main loop checking power status every second

- **handle_shutdown_signal()**: Responds to UPS hardware shutdown signal
- **initiate_shutdown()**: Performs graceful system shutdown
- **update_power_state()**: Determines current power state
- **start_shutdown_timer()**: Begins 30-second countdown after power loss

4. Communication Methods

- **GPIO (Required)**: Single-wire communication on GPIO18 for shutdown signal
- **UART (Optional)**: Serial communication for battery percentage, voltage monitoring

Implementation Steps

1. Hardware Setup

Required Connections

```
Raspberry Pi 4        UPSPACK_V3
--------------        -----------
GPIO18 (Pin 12)  <-->  STA (Signal)
GND (Pin 6)      <-->  GND
5V (Pin 2,4)     <-->  5V Output
```

Optional UART Connections (for detailed monitoring)

```
Raspberry Pi 4        UPSPACK_V3
--------------        -----------
TX (GPIO14)      <-->  RX
RX (GPIO15)      <-->  TX
GND              <-->  GND
```

2. Software Installation

Step 1: Enable Required Interfaces

bash

```
# Enable UART (optional, for detailed monitoring)
sudo raspi-config
# Navigate to: Interface Options -> Serial Port
# - Login shell: NO
# - Serial hardware: YES

# Add to /boot/config.txt (for UART)
echo "enable_uart=1" | sudo tee -a /boot/config.txt
echo "dtoverlay=pi3-miniuart-bt" | sudo tee -a /boot/config.txt
```

Step 2: Install Dependencies

bash

```
# Update system
sudo apt-get update
sudo apt-get upgrade -y

# Install Python dependencies
sudo apt-get install python3-pip python3-serial python3-rpi.gpio -y
pip3 install pyserial
```

Step 3: Copy Script

bash

```
# Create directory for UPS management
sudo mkdir -p /opt/ups_manager

# Copy the script (save as ups_manager.py)
sudo nano /opt/ups_manager/ups_manager.py
# Paste the Python script here

# Make executable
sudo chmod +x /opt/ups_manager/ups_manager.py
```

Step 4: Create Systemd Service

bash

```
# Create service file
sudo nano /etc/systemd/system/ups-manager.service
```

Add the following content:

```ini
[Unit]
Description=UPS Power Manager for Car Infotainment
After=multi-user.target

[Service]
Type=simple
ExecStart=/usr/bin/python3 /opt/ups_manager/ups_manager.py
Restart=always
RestartSec=10
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

Step 5: Enable and Start Service

```bash
# Reload systemd
sudo systemctl daemon-reload

# Enable service to start on boot
sudo systemctl enable ups-manager.service

# Start the service
sudo systemctl start ups-manager.service

# Check status
sudo systemctl status ups-manager.service
```

3. Integration with Infotainment System

Monitoring Files

The script creates notification files that your react-carplay system can monitor:

- `/tmp/ups_low_battery` - Created when battery is low
- `/tmp/ups_shutdown` - Created when shutdown is initiated

React Integration Example

```javascript
// Check for UPS warnings in your React app
const checkUPSStatus = () => {
  fetch('/tmp/ups_low_battery')
    .then(response => {
      if (response.ok) {
        // Display low battery warning
        showNotification('Low battery - car will shutdown soon');
      }
    })
    .catch(() => {/* File doesn't exist - normal operation */});
};

// Check every 5 seconds
setInterval(checkUPSStatus, 5000);
```

Common Issues and Troubleshooting

Issue 1: Script Not Starting on Boot

Symptom: Service fails to start automatically

Solutions:

```bash
# Check service logs
sudo journalctl -u ups-manager.service -n 50

# Verify script permissions
ls -la /opt/ups_manager/ups_manager.py

# Check if running as root
sudo systemctl status ups-manager.service
```

Issue 2: GPIO Permission Errors

Symptom: "GPIO setup failed" in logs

Solutions:

```bash
# Ensure script runs as root
# Check if GPIO is already in use
gpio readall

# Add user to gpio group (if not running as root)
sudo usermod -a -G gpio $USER
```

Issue 3: UART Communication Failure

Symptom: "UART setup failed" warning

Solutions:

```bash
# Verify serial port exists
ls -l /dev/ttyAMA0
ls -l /dev/serial0

# Check serial port mapping
ls -l /dev | grep serial

# Test with minicom
sudo apt-get install minicom -y
sudo minicom -D /dev/ttyAMA0 -b 9600
```

Issue 4: Immediate Shutdown After Power Loss

Symptom: System shuts down immediately instead of waiting

Possible Causes:

- Battery is already low
- UPS sending immediate shutdown signal
- GPIO bounce detection issue

Solutions:

```python
```

```
# Increase bounce time in GPIO setup (line in script)
GPIO.add_event_detect(SHUTDOWN_PIN, GPIO.FALLING,
                      callback=self.handle_shutdown_signal,
                      bouncetime=500)  # Increase from 300 to 500

# Increase shutdown delay
SHUTDOWN_DELAY = 60  # Increase from 30 to 60 seconds
```

Issue 5: System Not Powering On Automatically

Symptom: Pi doesn't start when car power returns

Solutions:

1. Verify UPS battery has sufficient charge (>30%)
2. Check UPS switch is in ON position
3. Ensure proper power connections to Pi GPIO or USB
4. Update UPS firmware if needed:

```bash
# Check UPS firmware version
sudo minicom -D /dev/ttyAMA0 -b 9600
# Look for "SmartUPS V3.x" in output
```

Issue 6: False Power Loss Detection

Symptom: System thinks power is lost when it isn't

Solutions:

```bash
# Check car voltage (should be 12-14V when running)
# Verify TYPE-C cable quality and connections
# Use thicker gauge wire for power connections
# Add capacitor across power input for filtering
```

Monitoring and Logs

View Real-time Logs

```bash
# Watch service logs
sudo journalctl -u ups-manager.service -f

# View log file
sudo tail -f /var/log/ups_manager.log

# Check system messages
dmesg | grep -i gpio
```

Log Rotation Setup

```bash
# Create logrotate config
sudo nano /etc/logrotate.d/ups-manager
```

Add:

```
/var/log/ups_manager.log {
    daily
    rotate 7
    compress
    delaycompress
    missingok
    notifempty
    create 644 root root
}
```

Performance Optimization

1. Reduce Power Consumption

```bash
```

```
# Disable unnecessary services
sudo systemctl disable bluetooth
sudo systemctl disable wifi

# Reduce CPU frequency when on battery
# Add to the Python script's battery power handler:
os.system("echo powersave | sudo tee /sys/devices/system/cpu/cpu0/cpufreq/scaling_governor")
```

2. Faster Boot Time

```bash
# Disable unnecessary boot services
sudo systemctl disable apt-daily.service
sudo systemctl disable apt-daily-upgrade.service

# Add to /boot/cmdline.txt
quiet fastboot
```

3. Auto-start Infotainment

```bash
# Add to /etc/rc.local before 'exit 0'
cd /home/pi/react-carplay && npm start &
```

Testing Procedures

1. Basic Functionality Test

```bash
# Run script manually
sudo python3 /opt/ups_manager/ups_manager.py

# In another terminal, monitor logs
sudo tail -f /var/log/ups_manager.log

# Disconnect power and observe behavior
```

2. Shutdown Timer Test

1. Disconnect car power

2. Monitor logs for "Starting 30 second shutdown timer"
3. Reconnect power before 30 seconds
4. Verify "Shutdown timer cancelled" in logs

3. Battery Level Test

```bash
# Simulate low battery (if UART connected)
# Monitor for low battery warnings at 10% and 5%
```

Safety Considerations

1. **Always test with non-critical data first**
2. **Ensure proper ventilation for UPS battery**
3. **Use appropriate fuses in car power connection**
4. **Mount UPS securely to prevent movement while driving**
5. **Regular battery health checks (every 6 months)**

Maintenance

Monthly

- Check log file size
- Verify automatic startup/shutdown
- Test UPS battery runtime

Quarterly

- Clean GPIO connections
- Update system packages
- Check for UPS firmware updates

Annually

- Replace UPS battery if needed
- Full system backup
- Review and update shutdown delays based on usage

Advanced Features

Email Notifications

Add email alerts for power events:

```python
import smtplib
from email.mime.text import MIMEText

def send_email_alert(subject, body):
    msg = MIMEText(body)
    msg['Subject'] = subject
    msg['From'] = 'pi@car.local'
    msg['To'] = 'your-email@example.com'

    s = smtplib.SMTP('localhost')
    s.send_message(msg)
    s.quit()
```

Remote Monitoring

Set up remote access to check car status:

```bash
# Install VPN for secure remote access
curl -L https://install.pivpn.io | bash

# Or use Tailscale for easier setup
curl -fsSL https://tailscale.com/install.sh | sh
```

Conclusion

This UPS management system provides reliable power control for your car infotainment system. Regular monitoring of logs and periodic testing ensures continued reliability. The modular design allows for easy customization based on your specific needs.